

CHƯƠNG 2

TRIỂN KHAI ỨNG DỤNG QUA DOCKER

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Hiểu về khái niệm Microservices và lợi ích của Containerization.
- Hiểu và mô tả được kiến trúc Docker và các thành phần chính.
- Thiết lập và xây dựng được Dockerfile tối ưu (multi-stage) và hiểu khái niệm về tối ưu hóa images.
- Cài đặt và cấu hình Docker trên Linux.
- Triển khai và đóng gói ứng dụng web trong Docker với Volume & Network.
- Áp dụng một số cơ chế bảo mật cơ bản cho Docker image.

2.1 Tổng quan về Docker và microservice

2.1.1 Khái niệm cơ bản về Docker

Trong bối cảnh phát triển phần mềm hiện đại, đặc biệt là kiến trúc Microservices, việc triển khai và quản lý các ứng dụng trở nên phức tạp hơn bao giờ hết. Mỗi microservice có thể được viết bằng ngôn ngữ lập trình khác nhau, phụ thuộc vào các thư viện và phiên bản hệ thống riêng. Khi chuyển từ môi trường phát triển (development) sang testing, staging rồi production, chúng ta thường gặp vấn đề “Works on my machine” – ứng dụng chạy tốt trên máy lập trình viên nhưng lỗi trên server.

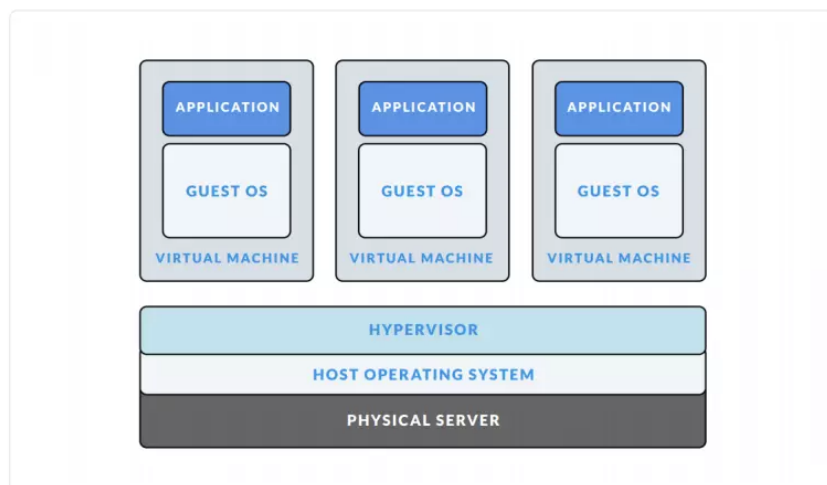
Trong thời gian đầu, mô hình máy chủ thường là một máy chủ vật lý, một hệ điều hành và chạy một ứng dụng.



Hình 2.1: Mô hình của một máy chủ vật lý

Tuy nhiên khi sự phát triển của công nghệ, đặc biệt là năng lực của phần cứng không ngừng tăng lên thì mô hình này sẽ sinh ra vấn đề là lãng phí tài nguyên khi mà hệ thống không thể sử dụng được hết lợi thế về mặt tài nguyên. Ngoài ra, còn có khó khăn trong việc mở rộng hệ thống (muốn mở rộng thì phải thuê thêm server, cấu hình, ...). Và từ đó, công nghệ ảo hóa (virtualization) được ra đời.

Virtualization (ảo hóa) là một công nghệ được thiết kế để tạo ra một tầng trung gian giữa phần cứng máy tính và phần mềm chạy trên nó. Ảo hóa được xây dựng dựa trên ý tưởng phân chia ổ đĩa, chúng phân chia máy chủ gốc thành nhiều máy chủ logic. Từ đó, một máy chủ vật lý đơn lẻ có thể tạo thành nhiều máy ảo chạy độc lập, riêng rẽ với nhau. Các máy ảo đó có thể có hệ điều hành riêng, ứng dụng riêng, độc lập với các máy ảo khác.



Hình 2.2: Mô hình hoạt động của công nghệ ảo hóa

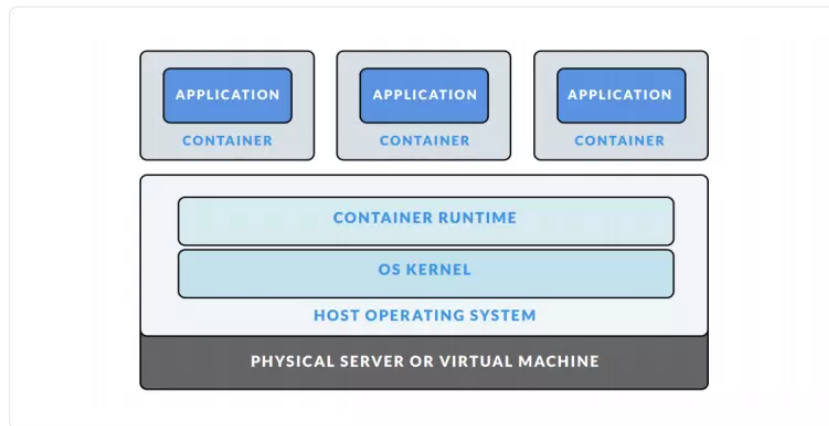
Sự ra đời của ảo hóa giúp cho một máy tính có thể chia ra thành nhiều máy ảo, từ đó tạo ra được nhiều môi trường khác nhau để chạy được nhiều ứng dụng. Vì vậy, năng suất của phần cứng được đẩy lên cao hơn, tận dụng được nhiều hơn. Tuy nhiên, việc ảo hóa này cũng nảy sinh ra nhiều vấn đề mới:

- **Ngốn tài nguyên:** Khi chạy một máy ảo, nó sẽ luôn chiếm một phần tài nguyên nhất định. Giả sử bạn có một máy chủ vật lý có 512GB SSD, 16GB RAM. Nếu tạo ra 2 máy ảo, mỗi máy cấp 128GB SSD và 4GB RAM thì máy chủ vật lý đã mất đi 256GB SSD và 8GB RAM mặc dù chỉ bật máy ảo lên và không sử dụng gì cả.
- **Tốn thời gian thực thi:** thời gian khởi động, restart, shutdown của các máy ảo cũng khá lâu, thường là vài phút.
- **Cồng kềnh:** server sẽ không thể chạy được hết hiệu suất khi mà phải chịu tải cho cả nhóm máy ảo, đồng thời cũng khó cho việc quản lý, mở rộng.

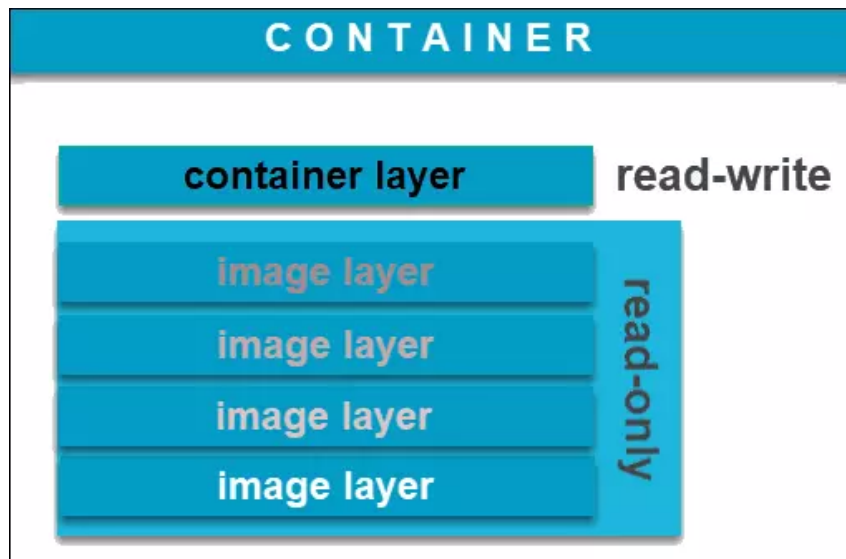
Để giải quyết các vấn đề trên, người ta đã phát minh ra Container, với công nghệ này, một máy chủ vật lý có thể sinh ra và chạy nhiều container tương tự như VM. Tuy nhiên, điểm đặc biệt ở đây là các container có thể dùng chung kernel và chia sẻ tài nguyên của máy chủ vật lý với nhau (CPU, RAM, ...) Vì thế tài nguyên sẽ được sử dụng hợp lý và linh động hơn, không bị gò bó trong một khoảng như VM.

Ngoài ra, việc container được chia ra thành nhiều layer khác nhau cũng khiến cho việc quản lý tài nguyên được hiệu quả hơn.

Cụ thể, container mới sẽ được xây dựng dựa trên các image layer dạng read-only. Trong mỗi container sẽ có thêm một layer với quyền read-write, mọi thay đổi của container sẽ chỉ được ghi vào đây. Như vậy, chỉ từ một vài image ban đầu chúng ta có thể tạo ra nhiều container khác nhau từ đó tiết kiệm được một phần không gian lưu trữ.



Hình 2.3: Cấu tạo của container



Hình 2.4: Cấu trúc layer trong container

Container nhẹ hơn Virtual Machine (VM) rất nhiều vì chia sẻ kernel của hệ điều hành host.

Lợi ích chính của Containerization (đặc biệt với Microservices):

- **Tính di động (Portability):** “Build once, run anywhere” – Chạy được trên laptop, server on-premise, cloud (AWS, Azure, GCP) mà không thay đổi.
- **Tính nhất quán (Consistency):** Môi trường giống hệt giữa dev, test, prod → Giảm lỗi môi trường.
- **Hiệu suất cao:** Khởi động nhanh (vài giây), tiêu tốn ít tài nguyên hơn VM (không cần emulate toàn bộ OS).
- **Dễ scale:** Dễ dàng scale horizontal (thêm nhiều instance) cho từng microservice.
- **Cô lập (Isolation):** Mỗi container có không gian process, filesystem riêng → Điều này giúp tăng cường khả năng bảo mật (hạn chế ảnh hưởng nếu một service bị tấn công).
- **CI/CD thân thiện:** Dễ tích hợp với Jenkins, GitHub Actions, Kubernetes.

Ví dụ thực tế: Netflix, Spotify, hay các ngân hàng Việt Nam sử dụng container để quản lý hàng nghìn microservices, đảm bảo uptime cao và deploy nhanh hàng ngày. Hay trong lĩnh vực về An toàn thông tin và quá trình nghiên cứu, thử nghiệm các giải pháp bảo mật, Docker giúp chúng ta triển khai các công cụ bảo mật (như vulnerability scanner, honeypot, SIEM agents) một cách nhanh chóng, lặp lại và cô lập.

So sánh Container so với Virtual Machine:

Bảng 2.1: So sánh Container và Virtual Machine

Tiêu chí	Virtual Machine (VM)	Container (Docker)
Kích thước	Lớn (GB – vài chục GB)	Nhỏ (MB – vài trăm MB)
Khởi động	Chậm (phút)	Nhanh (giây)
Hiệu suất	Thấp hơn (emulate hardware)	Cao (chia sẻ kernel host)
Số lượng trên 1 host	Ít	Rất nhiều (hàng trăm)
Cấp độ ảo hóa	Hardware/OS	OS-level (process isolation)

Ta có thể hình dung: VM như một ngôi nhà riêng biệt hoàn chỉnh. Container như các căn hộ trong chung cư – chia sẻ nền tảng (kernel) nhưng có không gian riêng.

Docker là nền tảng mã nguồn mở phổ biến nhất cho containerization. Docker cho phép đóng gói toàn bộ ứng dụng cùng dependencies (thư viện, cấu hình, runtime) vào một đơn vị chuẩn hóa gọi là container. Docker cung cấp cho người sử dụng những công cụ để có thể đóng gói, vận chuyển và chạy container một cách đơn giản và dễ dàng trên các nền tảng khác nhau một cách nhanh nhất với tiêu chí – “Build once, run anywhere”.



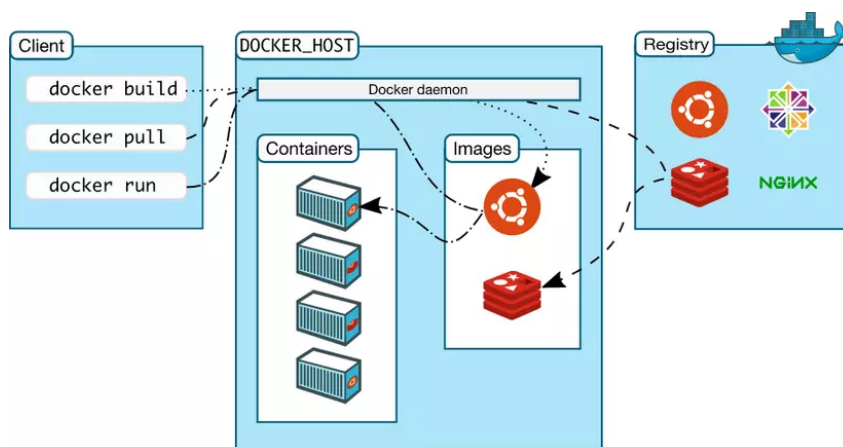
Hình 2.5: Ứng dụng Docker

Docker thực hiện ảo hóa ở mức hệ điều hành. Mỗi container là cô lập (isolated) với nhau nhưng đều dùng chung một số bin/lib và kernel của Host OS. Docker có thể làm việc trên nhiều nền tảng khác nhau như Linux, Microsoft Windows và Apple OS X. Ngoài ra, Docker còn hỗ trợ nhiều dịch vụ điện toán đám mây nổi tiếng như Microsoft Azure hay Amazon Web Services.

2.1.2 Kiến trúc của Docker

Docker hoạt động theo mô hình Client-Server, bao gồm Docker Engine (Daemon và CLI), Images, Containers, Registry và các thành phần hỗ trợ như Volumes/Networking. Docker sử dụng kỹ thuật ảo hóa cấp hệ điều hành (Namespace/Cgroups) để cô lập các ứng dụng trong các Container nhẹ, hiệu suất cao. Hiện nay Docker phân phối dưới các phiên bản phổ biến:

- **Docker Community Edition (CE):** là phiên bản miễn phí và chủ yếu dựa vào các sản phẩm nguồn mở khác.
- **Docker Enterprise (EE):** phiên bản dành cho các doanh nghiệp, khi sử dụng phiên bản này sẽ nhận được sự support của nhà phát hành, ngoài ra còn có thêm các tính năng quản lý và bảo mật.



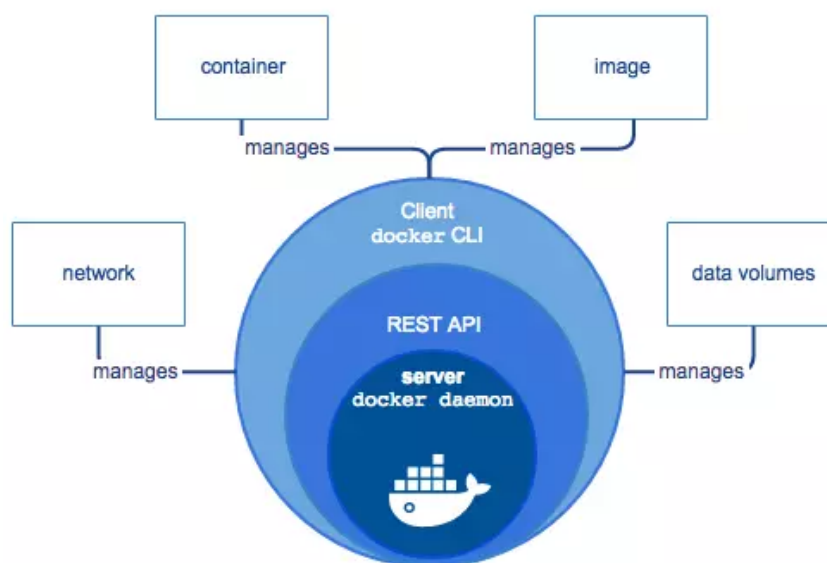
Hình 2.6: Kiến trúc của Docker

Kiến trúc tổng quan (Client-Server):

- **Docker Client (CLI):** cung cấp giao diện dòng lệnh (command line) (VD: `docker build`, `docker run`), cho người sử dụng, đồng thời cũng gửi request đến Docker daemon.
- **REST API:** Giao diện cho phép Client giao tiếp với Daemon.
- **Docker Daemon (dockerd):** "Trái tim" của Docker, chạy ngầm trên máy chủ đóng vai trò là server, quản lý các đối tượng: Image, Container, Network, Volume, nhận các RESTful request từ Docker Client và thực thi nó. Là một lightweight runtime giúp build, run và quản lý các containers và các thành phần liên quan khác.

Docker Daemon quản lý 4 đối tượng chính: image, container, network, volume.

- **Docker Image:** Đây là các bản mẫu (template) chỉ đọc (read-only) chứa mã nguồn, thư viện, và cấu hình để chạy ứng dụng. Image được xây dựng theo từng lớp (layers).



Hình 2.7: Các thành phần bên trong của Docker

- **Docker Container:** Phiên bản thực thi (instance) của Image. Nó là một tiến trình cô lập, nhẹ và có thể khởi chạy, dừng, xóa.
- **Docker Volumes:** Thành phần lưu trữ dữ liệu bền vững (persistent data), tách biệt khỏi container để dữ liệu không bị mất khi xóa container.
- **Docker Network:** Cấu hình mạng giúp các container giao tiếp với nhau hoặc với Internet.

Ngoài ra còn có các thành phần khác thường gặp như:

- **Dockerfile:** Tập văn bản chứa các chỉ dẫn (lệnh) để tự động hóa quy trình build Docker Image.
- **Docker Registry (Hub):** Nơi lưu trữ và chia sẻ Docker Image (VD: Docker Hub, Private Registry).

Các thao tác hoạt động chung khi sử dụng Docker đó là:

- **Build:** Người dùng viết cấu hình thiết lập trong file Dockerfile và dùng Docker Client để gửi lệnh build Image.
- **Push/Pull:** Image được lưu vào Registry.
- **Run:** Docker Daemon lấy image và khởi chạy Container.
- Cấu hình tối thiểu để chạy Docker: CPU 1-2 Cores, RAM tối thiểu 1GB (khuyến nghị ≥ 2 GB).

2.1.3 Khái niệm về microservice và ứng dụng docker

Trong phát triển phần mềm truyền thống, hầu hết các ứng dụng được xây dựng theo kiến trúc Monolithic (đơn khối) – tất cả các chức năng (authentication, business logic, database access, UI...) nằm trong một codebase duy nhất và được triển khai như một đơn vị thống nhất.

Khi lập trình viên biên dịch và triển khai, toàn bộ hệ thống được đóng gói thành

một tệp duy nhất (như .jar, .war, hoặc .exe) và chạy trên cùng một môi trường. Mọi chức năng, từ đăng nhập, thanh toán đến xử lý dữ liệu, đều phụ thuộc và hoạt động chung trong ứng dụng đó.

Đặc điểm của kiến trúc Monolithic

Kiến trúc Monolithic có một số đặc trưng nổi bật giúp nhận diện rõ ràng:

- Mọi module được gắn kết chặt chẽ: Các phần của ứng dụng không thể chạy độc lập, mà phụ thuộc lẫn nhau.
- Triển khai đồng nhất: Toàn bộ ứng dụng được build, test và deploy cùng lúc.
- Cơ sở dữ liệu duy nhất: Các chức năng cùng truy cập và thao tác trên một nguồn dữ liệu chung.
- Cập nhật đồng bộ: Mỗi khi có thay đổi nhỏ, hệ thống phải được build và triển khai lại toàn bộ.

Nhờ tính đơn khối, kiến trúc Monolithic giúp việc phát triển ban đầu dễ dàng hơn, nhưng về sau lại có thể gây ra thách thức lớn khi hệ thống mở rộng.

Những hạn chế:

- Hạn chế về khả năng mở rộng – Việc mở rộng đòi hỏi phải sao chép toàn bộ ứng dụng, gây lãng phí tài nguyên.
- Tốc độ phát triển chậm – Khi mã nguồn lớn lên, việc thêm tính năng mới trở nên phức tạp.
- Khó bảo trì – Một mã nguồn lớn dần trở nên khó quản lý theo thời gian.
- Khó thay đổi công nghệ – Việc áp dụng công nghệ mới rất khó khăn mà không viết lại toàn bộ hệ thống.

Khi ứng dụng lớn lên (hàng trăm nghìn dòng code), việc bảo trì, mở rộng và cập nhật trở nên cực kỳ khó khăn. Một thay đổi nhỏ ở module thanh toán có thể làm toàn bộ hệ thống downtime. Đây chính là lý do Microservices Architecture ra đời.

Kiến trúc Microservices chia nhỏ một ứng dụng thành các dịch vụ nhỏ, độc lập, giao tiếp với nhau qua API. Mỗi dịch vụ chịu trách nhiệm cho một chức năng cụ thể và có thể được phát triển, triển khai, cũng như mở rộng một cách độc lập. Microservices là phong cách kiến trúc mà một ứng dụng lớn được xây dựng từ nhiều dịch vụ nhỏ (services), mỗi service:

- Chịu trách nhiệm cho một chức năng kinh doanh cụ thể (ví dụ: User Service, Payment Service, Notification Service...).
- Có codebase riêng, có thể dùng ngôn ngữ lập trình và database khác nhau.
- Triển khai độc lập và giao tiếp với nhau qua API nhẹ (thường là RESTful API, gRPC, Message Queue như Kafka).

Ví dụ thực tế: Một hệ thống ngân hàng trực tuyến có thể gồm:

- User Service (quản lý tài khoản)
- Loan Service (xét duyệt vay)
- Transaction Service (chuyển tiền)
- Notification Service (gửi SMS/Email)

Như vậy, mỗi service trong hệ thống sẽ chạy độc lập và scale riêng.

So sánh giữa Monolithic vs Microservices:

Bảng 2.2: So sánh Monolithic và Microservices

Tiêu chí	Monolithic Architecture	Microservices Architecture
Cấu trúc	Một codebase lớn, tightly coupled	Nhiều service nhỏ, loosely coupled
Triển khai	Triển khai toàn bộ ứng dụng	Triển khai từng service độc lập
Scale	Scale toàn bộ (scale monolithic)	Scale từng service (horizontal scaling)
Công nghệ	Thường dùng chung một stack	Mỗi service chọn tech stack phù hợp
Fault Isolation	Một lỗi có thể làm sập toàn bộ hệ thống	Lỗi ở một service không ảnh hưởng toàn bộ
Tốc độ phát triển	Ban đầu nhanh, sau chậm	Ban đầu phức tạp, sau nhanh và linh hoạt
Team organization	Team lớn quản lý toàn bộ	Nhiều team nhỏ (2-pizza team) tự chủ

Lợi ích chính của Microservices:

- **Tốc độ phát triển & Deploy nhanh:** Đội ngũ có thể deploy service mới nhiều lần mỗi ngày mà không ảnh hưởng service khác.
- **Scalability linh hoạt:** Service thanh toán có thể scale lên 50 instances vào giờ cao điểm, trong khi service báo cáo chỉ cần 2 instances.
- **Công nghệ đa dạng:** Dùng Java cho service core, Python cho AI recommendation, Node.js cho notification.
- **Fault tolerance & Resilience:** Hệ thống vẫn hoạt động khi một service bị lỗi.
- **Dễ bảo trì và test:** Codebase nhỏ → dễ hiểu, dễ test.

Tuy nhiên, các thách thức (Challenges) có thể gặp phải là:

- **Phức tạp phân tán:** Phải quản lý networking, service discovery, load balancing, distributed tracing.
- **Data consistency:** Mỗi service có database riêng → vấn đề eventual consistency.
- **Overhead vận hành:** Monitoring, logging, security cho hàng trăm service.
- **Latency mạng:** Gọi API giữa các service có thể chậm hơn function call trong monolith.

Docker – “Người bạn đồng hành hoàn hảo” của Microservices

Containerization (đặc biệt là Docker) giúp các nhà phát triển giải quyết hầu hết các thách thức khi triển khai Microservices. Vai trò của Docker trong xây dựng phần mềm theo kiến trúc Microservices có thể thấy:

- **Đóng gói và cô lập:** Mỗi microservice được đóng gói thành một Docker Image riêng → Container chạy độc lập, có môi trường runtime giống hệt.
- **Tính nhất quán môi trường:** Giải quyết vấn đề “Works on my machine”.
- **Triển khai nhanh & Scale dễ dàng:** Khởi động container chỉ trong vài giây, dễ scale horizontal.
- **Tiết kiệm tài nguyên:** Nhẹ hơn VM rất nhiều → một server vật lý/cloud instance có thể chạy hàng chục/hàng trăm containers.
- **CI/CD thân thiện:** Dễ tích hợp vào pipeline (build → test → push image → deploy).
- **Bảo mật tốt hơn** (từ góc nhìn An toàn thông tin):

- Cô lập nhờ Linux namespaces & cgroups.
- Mỗi dịch vụ có attack surface riêng.
- Dễ áp dụng nguyên tắc đặc quyền tối thiểu.
- Dễ quét các image để phát hiện lỗ hổng.
- Công cụ Docker Compose giúp định nghĩa và chạy nhiều container cùng lúc (multi-container app) qua file `docker-compose.yml` – rất phù hợp để phát triển và kiểm thử trên local một hệ thống microservices nhỏ.
- Kubernetes (K8s) thường được dùng cùng Docker để tự động hóa các quy trình (quản lý, scale, tự healing) hàng trăm các microservices khác nhau ở chế độ production.

2.1.4 Dockerfile và cú pháp xây dựng

Dockerfile là một file văn bản (text file) không có phần mở rộng hoặc có đuôi `.Dockerfile`, chứa tập hợp các chỉ lệnh (instructions) để Docker Daemon xây dựng nên một Docker Image.

Nói cách khác: Dockerfile giống như “bài hướng dẫn chi tiết để nấu bữa ăn” – trong đó ghi lại từng bước một cách rõ ràng, Docker sẽ thực hiện theo thứ tự để tạo ra một “món ăn” (có thể coi là Image) có thể dùng để chạy Container.

Đặc điểm quan trọng đối file dockerfile là:

- Dockerfile sử dụng cú pháp đơn giản, dễ đọc.
- Mỗi chỉ lệnh tạo ra một layer trong Image.
- Image được build theo thứ tự từ trên xuống dưới.
- File Dockerfile thường đặt ở thư mục gốc (root) của dự án.

Những lợi ích khi sử dụng Dockerfile:

- Quản trị viên hệ thống có thể tự động hóa việc xây dựng Image.
- Dễ dàng quản lý phiên bản (đặt trong Git).
- Đảm bảo tính lặp lại khi xây dựng hoặc tạo Image (reproducible builds).
- Dễ chia sẻ và hợp tác trong team.
- **Lưu ý:** Việc cài đặt dịch vụ Docker trên máy tính cá nhân có thể tự tìm hiểu và thực hiện dưới dạng nội dung tự học ở nhà, không triển khai cài đặt trên lớp.

Quy trình xây dựng Image cơ bản với docker như sau:

Sử dụng lệnh: `$docker build -t ten-image:tag .`

Trong đó:

- `-t`: là tag được sử dụng để đánh dấu, phân loại thẻ, phiên bản cho Image.
- `.`: Thư mục hiện hành chứa dockerfile và các tập tin cần Copy.

Khi thực hiện lệnh, Docker Daemon sẽ đọc nội dung của Dockerfile từ trên xuống, thực thi từng lệnh và cache layer.

Lưu ý: Có thể sử dụng file `.dockerignore` để loại trừ các file không cần thiết (`node_modules`, `.git`, `*.log...`) trong thư mục chứa dockerfile, điều này sẽ giúp quản lý tài nguyên dễ dàng và logic.

Các chỉ lệnh (Instructions) cơ bản và quan trọng nhất:

Bảng 2.3: Các chỉ lệnh Dockerfile cơ bản

Chỉ lệnh	Mô tả	Ví dụ	Ghi chú quan trọng
FROM	Chỉ định image cơ sở (base image)	FROM node:20-alpine	Phải là lệnh đầu tiên
LABEL	Thêm metadata (tác giả, version, description)	LABEL maintainer="yourname@email.com"	Tốt cho documentation
ARG	Định nghĩa biến build-time (có thể truyền từ command line)	ARG NODE_ENV=production	Không tồn tại trong runtime
ENV	Đặt biến môi trường (có trong runtime)	ENV NODE_ENV=production	Dùng cho ứng dụng
WORKDIR	Thiết lập thư mục làm việc mặc định	WORKDIR /app	Nên dùng sớm
COPY	Copy file/thư mục từ host vào image	COPY . .	Tốt hơn ADD
ADD	Copy + hỗ trợ remote URL và tự giải nén tar	ADD app.tar.gz /app/	Chỉ dùng khi cần
RUN	Thực thi lệnh trong quá trình build	RUN apk add --no-cache python3	Tạo layer mới
EXPOSE	Khai báo port mà container sẽ lắng nghe	EXPOSE 3000	Chỉ là documentation
CMD	Lệnh mặc định chạy khi container start (có thể override)	CMD ["npm", "start"]	Chỉ dùng 1 CMD
ENTRYPOINT	Lệnh chính, khó override hơn CMD	ENTRYPOINT ["docker-entrypoint.sh"]	Thường kết hợp CMD
USER	Chạy container với user không phải root	USER node	Tăng bảo mật
VOLUME	Khai báo volume mount point	VOLUME /app/data	Dữ liệu bền vững
HEALTHCHECK	Kiểm tra sức khỏe của container	HEALTHCHECK CMD curl -f http://localhost	Rất quan trọng trong chế độ production

Ví dụ xây dựng dockerfile cho ứng dụng Nodejs như sau:

```
# Stage 1: Build (sẽ tối ưu sau)
FROM node:20-alpine AS builder

WORKDIR /app

# Copy package files trước để tận dụng cache layer
COPY package*.json ./
RUN npm ci --only=production

# Copy source code
COPY . .

# Stage 2: Production (sẽ học chi tiết ở phần Multi-stage)
FROM node:20-alpine
# Chỉ định thư mục làm việc chính
WORKDIR /app
```

```
# Copy từ builder
COPY --from=builder /app /app

# Security: chạy với user không root
USER node
# Cổng mặc định sẽ hoạt động
EXPOSE 3000

ENV NODE_ENV=production
# Thực thi lệnh
CMD ["node", "server.js"]
```

Một số lưu ý khi tạo dockerfile:

- Nên sử dụng các base image nhẹ: Ưu tiên alpine, slim, hoặc distroless thay vì image đầy đủ (ubuntu, debian).
- Tận dụng Layer Caching:
 - Copy package.json trước khi copy toàn bộ source code.
 - Đặt lệnh thay đổi ít (ENV, WORKDIR) lên trên, lệnh thay đổi nhiều (COPY source, RUN npm install) xuống dưới.
- Giảm kích thước Image:
 - Kết hợp nhiều lệnh RUN bằng dấu gạch chéo ngược (\) và &&.
 - Xóa cache, file tạm sau khi install (VD: rm -rf /var/cache/apk/*).
- Tăng tính bảo mật bằng cách:
 - Không chạy với tài khoản root, tạo tài khoản người dùng thường.
 - Không gán cứng các mã code quan trọng (như password, API key) → thay vào đó hãy dùng ARG/ENV hoặc Docker Secret.
 - Thực hiện quét image bằng Trivy hoặc Docker Scout trước khi deploy.

Nội dung file .dockerignore tham khảo:

```
node_modules
.git
*.log
Dockerfile
README.md
```

Sau khi tạo xong nội dung cho dockerfile, tiếp theo ta cần thực hiện lệnh build để tạo Image theo cú pháp sau:

```
# Build với tag
$docker build -t myapp:v1.0 .
# Build không cache (khi cần rebuild sạch)
$docker build --no-cache -t myapp:v1.0 .
# Xem layers của image
$docker history myapp:v1.0
```

Kết thúc quá trình build, nếu không xảy ra lỗi thì chúng ta đã tạo ra một image mới, để kiểm tra danh sách các image trên hệ thống, ta thực hiện lệnh sau:

```
$docker images
```



Hình 2.8: Danh sách các Image có trên hệ thống và được quản lý bởi Docker

2.2 Triển khai ứng dụng với Docker

2.2.1 Quản lý container và image

Sau khi đã xây dựng được Dockerfile và tạo ra Docker Image, bước tiếp theo và rất quan trọng là quản lý Image và Container một cách hiệu quả.

Trong môi trường thực tế, một dự án có thể cần phải được kiểm thử qua nhiều giai đoạn khác nhau (như: development, testing, production), khi đó, chúng ta sẽ phải làm việc với hàng chục đến hàng trăm images và containers. Việc quản lý tốt giúp:

- Tiết kiệm dung lượng ổ cứng
- Tăng tốc độ làm việc
- Phát hiện và xử lý sự cố nhanh chóng
- Đảm bảo môi trường sạch sẽ và bảo mật

Nội dung phần này tập trung vào các lệnh Docker CLI quan trọng nhất để quản lý Image và Container.

Quản lý Docker Images. Image là template chỉ đọc dùng để tạo Container, chúng được tạo ra từ dockerfile. Các lệnh thường sử dụng để quản lý Image trên CLI bao gồm:

Bảng 2.4: Các lệnh quản lý Docker Image

Lệnh	Mô tả	Ví dụ sử dụng
<code>docker images</code> / <code>docker image ls</code>	Liệt kê các images hiện có	<code>docker images -a</code> (hiển thị tất cả)
<code>docker pull</code>	Tải image từ Registry	<code>docker pull nginx:alpine</code>
<code>docker build</code>	Xây dựng image từ Dockerfile	<code>docker build -t myapp:1.0 .</code>
<code>docker tag</code>	Đổi tên/tag cho image	<code>docker tag myapp:1.0 myregistry.com/myapp:1.0</code>
<code>docker push</code>	Đẩy image lên Registry (Docker Hub, Harbor...)	<code>docker push myregistry.com/myapp:1.0</code>
<code>docker rmi</code>	Xóa image	<code>docker rmi nginx:alpine</code>
<code>docker image prune</code>	Xóa images không dùng (dangling)	<code>docker image prune -a --force</code>
<code>docker save</code> / <code>docker load</code>	Lưu và nạp image dưới dạng file .tar	<code>docker save -o myapp.tar myapp:1.0</code>

Một số lưu ý quan trọng khi quản lý Image như sau:

- Luôn dùng tag rõ ràng (ví dụ: v1.2.3, latest, production).

Chương 2. Triển khai ứng dụng qua Docker

- Tránh dùng `latest` trong production vì khó kiểm soát phiên bản.
- Thường xuyên dọn dẹp: `$docker image prune -a` để xóa images không dùng.
- Sử dụng lệnh `$docker system df` để xem dung lượng đang chiếm của Images, Containers, Volumes.

Ví dụ:

```
# Xây dựng và tag image
docker build -t webapp:1.0 .
docker tag webapp:1.0 mycompany/webapp:1.0
# Liệt kê các Image
docker images
# Xóa image cũ
docker rmi webapp:0.9
```

Quản lý các Container. Container được coi là instance đang chạy (hoặc đã dừng) của một Image. Trong quá trình tạo và sử dụng các container, chúng ta sẽ thường xuyên sử dụng đến các câu lệnh CLI sau:

Bảng 2.5: Các lệnh quản lý Docker Container

Lệnh	Mô tả	Ví dụ
<code>docker run</code>	Tạo và chạy container mới	<code>docker run -d -p 8080:80 nginx:alpine</code>
<code>docker ps</code> / <code>docker container ls</code>	Liệt kê container đang chạy	<code>docker ps -a</code> (hiển thị tất cả)
<code>docker start</code> / <code>stop</code> / <code>restart</code>	Khởi động / dừng / khởi động lại	<code>docker stop myweb</code>
<code>docker pause</code> / <code>unpause</code>	Tạm dừng / tiếp tục container	<code>docker pause myweb</code>
<code>docker rm</code>	Xóa container	<code>docker rm -f myweb</code> (-f: force)
<code>docker logs</code>	Xem log của container	<code>docker logs -f myweb</code> (-f: follow)
<code>docker exec</code>	Chạy lệnh bên trong container đang chạy	<code>docker exec -it myweb sh</code>
<code>docker inspect</code>	Xem thông tin chi tiết container	<code>docker inspect myweb</code>
<code>docker stats</code>	Xem tài nguyên sử dụng (CPU, RAM, Network)	<code>docker stats</code>
<code>docker cp</code>	Copy file giữa host và container	<code>docker cp myweb:/app/config ./</code>
<code>docker rename</code>	Đổi tên container	<code>docker rename oldname newname</code>

Ví dụ thực tế đầy đủ như sau:

```
$docker run -d --name myapi \
-p 3000:3000 \
-e NODE_ENV=production \
-v /data/logs:/app/logs \
```

```
--restart=unless-stopped \
myapi:1.2
```

Ý nghĩa của các tham số trong câu lệnh trên:

Bảng 2.6: Ý nghĩa các tham số của lệnh `docker run`

Tham số	Ý nghĩa	Giải thích ý nghĩa
<code>docker run</code>	Lệnh tạo và khởi chạy một container mới từ Image	Đây là lệnh cốt lõi. Docker sẽ tạo container và start ngay lập tức.
<code>-d</code>	Detached mode (chạy ngầm)	Container chạy ở background. Nếu không có <code>-d</code> , container sẽ chạy foreground (chiếm terminal).
<code>--name myapi</code>	Đặt tên cho container	Đặt tên để nhớ là myapi. Nếu không đặt, Docker sẽ tự tạo tên ngẫu nhiên (ví dụ: happy_fermi).
<code>-p 3000:3000</code>	Port Mapping (Ánh xạ cổng)	<code>-p <host_port>:<container_port></code> Port 3000 trên máy host (máy thật/cloud) được ánh xạ vào port 3000 bên trong container.
<code>-e</code> <code>NODE_ENV=production</code>	Environment Variable (Biến môi trường)	Truyền biến môi trường vào container. Ứng dụng Node.js sẽ đọc được <code>process.env.NODE_ENV = "production"</code> .
<code>-v</code> <code>/data/logs:/app/logs</code>	Volume Mount (Ánh xạ thư mục)	Mount thư mục <code>/data/logs</code> trên host vào thư mục <code>/app/logs</code> bên trong container. → Dữ liệu log sẽ được lưu bền vững, không mất khi container bị xóa.
<code>--restart=unless-stopped</code>	Restart Policy (Chính sách khởi động lại)	Container sẽ tự động restart nếu bị chết, trừ khi bạn chủ động dừng bằng lệnh docker stop . Các policy phổ biến: no , always , on-failure , unless-stopped .
<code>myapi:1.2</code>	Image name + Tag	Chỉ định Image sẽ dùng để tạo container. Ở đây là image tên myapi, tag 1.2.

2.2.2 Quản lý Volumn trong docker

Mặc định, tất cả các file được tạo bên trong một container đều được lưu trữ trên container layer có thể ghi (writable container layer). Layer này nằm trên các image layer chỉ đọc (read-only) và bất biến (immutable).

Dữ liệu được ghi vào container layer không bền vững – khi container bị hủy (destroyed), toàn bộ dữ liệu trong layer này sẽ mất. Điều này gây khó khăn khi cần trích xuất dữ liệu để sử dụng cho các tiến trình khác.

Mỗi container sở hữu một writable layer riêng biệt. Do đó, việc trích xuất dữ liệu từ layer này ra host hoặc sang container khác không đơn giản và không được hỗ trợ trực tiếp.

Các tùy chọn mount lưu trữ (Storage Mount Options). Để lưu trữ dữ liệu bên ngoài writable layer của container, Docker hỗ trợ các loại storage mount sau:

- Volume mounts
- Bind mounts
- tmpfs mounts
- Named pipes

Dù sử dụng loại mount nào, dữ liệu từ bên trong container đều được nhìn nhận như nhau: nó xuất hiện dưới dạng thư mục hoặc tệp tin trong hệ thống file của container. Cụ thể như sau:

Volume mounts (Khối lưu trữ): Volume là cơ chế lưu trữ bền vững được quản lý bởi Docker daemon. Dữ liệu trong volume vẫn được giữ nguyên ngay cả khi container sử dụng nó đã bị xóa.

- Dữ liệu volume được lưu trên hệ thống file của host.
- Để tương tác với dữ liệu, phải mount volume vào container.
- Việc truy cập trực tiếp vào dữ liệu volume từ host (không qua mount) là hành vi không được hỗ trợ, có thể dẫn đến hỏng hóc dữ liệu hoặc volume một cách không mong muốn.

Ưu điểm nổi bật khi sử dụng volume: Volume rất phù hợp cho các ứng dụng yêu cầu hiệu suất cao (performance-critical) và nhu cầu lưu trữ dài hạn. Vì vị trí lưu trữ được quản lý bởi daemon, volume cung cấp hiệu năng truy xuất file thô tương đương với việc truy cập trực tiếp hệ thống file của host.

Bind Mounts (Mount ràng buộc). Bind mount tạo ra một liên kết trực tiếp giữa một đường dẫn trên hệ thống host và container. Nó cho phép container truy cập file hoặc thư mục ở bất kỳ vị trí nào trên host. Đặc điểm đó là:

- Bind mount không được Docker cô lập, do đó cả tiến trình Docker và các tiến trình không phải Docker trên host đều có thể đồng thời sửa đổi các file được mount.
- Chỉ nên sử dụng bind mount khi cần truy cập dữ liệu từ cả hai phía (container và host) một cách linh hoạt.

c. tmpfs Mounts (Temporary File System). tmpfs mount lưu trữ file trực tiếp trong bộ nhớ RAM của máy host, không ghi dữ liệu xuống ổ cứng. Đây là loại lưu trữ tạm thời (ephemeral): dữ liệu sẽ mất khi container dừng hoặc khởi động lại, hoặc khi host reboot. tmpfs không lưu trữ bền vững trên Docker host cũng như trong filesystem của container.

Ứng dụng phù hợp: Sử dụng tmpfs khi cần lưu trữ tạm thời trong bộ nhớ, chẳng hạn như:

- Cache dữ liệu trung gian
- Xử lý thông tin nhạy cảm (ví dụ: credentials, mật khẩu)
- Giảm thiểu I/O ổ đĩa

Chỉ nên sử dụng tmpfs khi dữ liệu không cần tồn tại sau phiên làm việc của container.

d. Named Pipes. Named pipes cho phép giao tiếp giữa Docker host và container thông qua cơ chế pipe có tên. Ứng dụng phổ biến: Chạy một công cụ bên thứ ba bên trong container và kết nối với Docker Engine API thông qua named pipe.

Trong nội dung môn học này, chúng ta sẽ tập trung vào việc sử dụng volume để lưu trữ dữ liệu của container.

Volume là các kho lưu trữ dữ liệu bền vững cho các container, được tạo và quản lý bởi Docker. Có thể tạo một volume bằng cách sử dụng lệnh `docker volume create`, hoặc Docker có thể tạo một volume trong quá trình tạo container hoặc service. Khi người dùng tạo một volume, nó được lưu trữ trong một thư mục trên Docker host. Khi chúng ta gắn (mount) volume vào một container, thư mục này chính là thứ được gắn vào container. Điều này tương tự như cách hoạt động của bind mount, ngoại trừ việc các volume được quản lý bởi Docker và được cách ly khỏi các chức năng cốt lõi của máy chủ.

Volume là cơ chế được ưu tiên để lưu trữ dữ liệu bền vững được tạo ra và sử dụng bởi các container Docker. Trong khi bind mount phụ thuộc vào cấu trúc thư mục và hệ điều hành của máy chủ, volume được quản lý hoàn toàn bởi Docker. Volume phù hợp trong cho các trường hợp sử dụng sau đây:

- Volume dễ sao lưu hoặc di chuyển hơn so với bind mount.
- Người dùng có thể quản lý volume bằng các lệnh Docker CLI hoặc Docker API.
- Volume hoạt động trên cả container Linux và Windows.
- Volume có thể được chia sẻ an toàn hơn giữa nhiều container.
- Các volume mới có thể được điền sẵn nội dung bởi một container hoặc quá trình build.
- Khi ứng dụng yêu cầu hiệu năng I/O cao.

Tuy nhiên, Volume không phải là lựa chọn tốt nếu ta cần truy cập các tệp từ máy chủ, vì volume được quản lý hoàn toàn bởi Docker. Hãy sử dụng bind mount nếu cần truy cập các tệp hoặc thư mục từ cả container và máy chủ.

Cú pháp để gắn một volume vào khi thực thi khởi tạo container như sau:

```
$docker run --mount type=volume,src=<volume-name>,dst=<mount-path>
$docker run --volume <volume-name>:<mount-path>
```

Trong câu lệnh `docker run` ở trên, có sử dụng 2 tùy chọn là `--mount` và `--volume`. Nhìn chung, nên sử dụng tùy chọn `--mount`, sự khác biệt chính là cờ `--mount` rõ ràng hơn và hỗ trợ tất cả các tùy chọn có sẵn. Và thường được sử dụng trong các trường hợp sau đây:

- Chỉ định các tùy chọn trình điều khiển ổ đĩa
- Gắn thư mục con của ổ đĩa
- Gắn ổ đĩa vào dịch vụ Swarm

Các tùy chọn khi sử dụng với `--mount` như sau:

Bảng 2.7: Tùy chọn của `--mount`

Tùy chọn	Mô tả
source, src	Nguồn của mount. Đối với named volume, đây là tên của volume. Đối với anonymous volume, trường này được bỏ qua.
destination, dst, target	Đường dẫn bên trong container nơi file hoặc thư mục sẽ được mount vào.
volume-subpath	Đường dẫn đến một thư mục con bên trong volume cần mount vào container. Thư mục con này phải tồn tại trong volume trước khi mount.

Chương 2. Triển khai ứng dụng qua Docker

Tùy chọn	Mô tả
readonly, ro	Nếu có tùy chọn này, volume sẽ được mount vào container ở chế độ chỉ đọc (read-only).
volume-nocopy	Nếu có tùy chọn này, dữ liệu hiện có tại đường dẫn đích (destination) sẽ không được sao chép vào volume khi volume đang trống. Mặc định, Docker sẽ sao chép dữ liệu từ container vào volume nếu volume rỗng.
volume-opt	Có thể chỉ định nhiều lần. Mỗi lần là một cặp key=value để truyền các tùy chọn bổ sung cho volume driver.

Ví dụ:

```
$docker run --mount type=volume,src=myvolume,dst=/data,ro,volume-subpath=/foo
```

Khi sử dụng với tùy chọn `--volume` hoặc `-v`, sẽ có 3 trường thông tin (ngăn cách nhau bởi dấu “:”) cần cung cấp để docker engine có thể hiểu và thực thi.

```
$docker run -v [<volume-name>:<mount-path>[:opts]]
```

Lưu ý thứ tự của các trường trên cần phải chính xác, đối với các volume được đặt tên, trường đầu tiên là tên của volume và là duy nhất trên một máy chủ nhất định. Đối với các volume ẩn danh, trường đầu tiên được bỏ qua. Trường thứ hai là đường dẫn nơi tệp hoặc thư mục được gắn kết trong container. Trường thứ ba là tùy chọn, và là một danh sách các tùy chọn được phân tách bằng dấu phẩy. Các tùy chọn hợp lệ cho `--volume` với ổ đĩa dữ liệu bao gồm:

Bảng 2.8: Tùy chọn của `--volume`

Tùy chọn	Mô tả
readonly, ro	Nếu có tùy chọn này, volume sẽ được mount vào container ở chế độ chỉ đọc (read-only).
volume-nocopy	Nếu có tùy chọn này, dữ liệu hiện có tại đường dẫn đích (destination) sẽ không được sao chép vào volume khi volume đang trống. Mặc định, Docker sẽ sao chép dữ liệu từ container vào volume nếu volume rỗng.

Ví dụ:

```
$docker run -v myvolume:/data:ro
$docker run -v myvolume:/data:ro --volume-opt=type=none mycontainer
```

Một số lệnh thường được sử dụng để quản lý và thao tác với volume:

```
$docker volume ls          # Liệt kê volumes
$docker volume create mydata # Tạo volume
$docker volume inspect mydata # Xem chi tiết
$docker volume rm mydata    # Xóa volume
$docker volume prune        # Xóa tất cả volume không sử dụng
```

Ví dụ sau sẽ khởi tạo container cho CSDL PostgreSQL và dữ liệu database sẽ được lưu trong docker volume có tên là `postgres_data`, khi container bị xóa thì dữ liệu này sẽ được bảo toàn không bị mất.

```
$docker run -d \
  --name mydb \
  -e POSTGRES_PASSWORD=Secret123 \
  -v postgres_data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:16-alpine
```

Một ví dụ khác được sử dụng với chế độ bind mount giành trong quá trình phát triển.

```
$docker run -d --name myapp \
  -v $(pwd)/logs:/app/logs -p 3000:3000 myapi:1.0
```

Đặc điểm của câu lệnh trên đó là dữ liệu trong container sẽ được ghi trực tiếp xuống thư mục của ổ đĩa, và người dùng hoàn toàn có thể tương tác, sử dụng, check logs trong quá trình phát triển ứng dụng.

2.2.3 Quản lý Network trong docker

Docker Network là một hệ thống mạng ảo được Docker thiết kế để cho phép các container giao tiếp với nhau hoặc với các dịch vụ bên ngoài. Mỗi container trong Docker chạy trên một lớp mạng riêng biệt, và nhờ có Docker Network, chúng có thể chia sẻ dữ liệu, gửi nhận tín hiệu hoặc tương tác qua các cổng dịch vụ một cách dễ dàng.

Docker mặc định tạo ra 3 network cơ bản khi cài đặt: bridge (default), host, và none. Ngoài ra, Docker còn cung cấp nhiều loại network khác như bridge, host, overlay, macvlan, giúp người dùng linh hoạt trong việc thiết lập môi trường phù hợp với từng kịch bản triển khai.

Docker Network cung cấp tính năng network isolation (cô lập mạng), DNS resolution tự động giữa các container và IPAM (IP Address Management) tích hợp. Trong các mô hình triển khai microservices, Kubernetes hay Docker Swarm, việc cấu hình Docker Network đúng cách là yếu tố then chốt để duy trì tính khả dụng và mở rộng của toàn bộ hạ tầng.

Mỗi Docker Network được vận hành thông qua một network driver – thành phần đóng vai trò quyết định cách các container giao tiếp với nhau và với bên ngoài. Mỗi loại driver mang lại một kiểu cấu trúc mạng khác nhau, phù hợp với từng mục đích sử dụng trong phát triển và triển khai hệ thống.

Các loại Docker Network driver phổ biến bao gồm:

- **Bridge (mặc định):** Cho phép các container trên cùng một host giao tiếp với nhau qua mạng ảo.
- **Host:** Container dùng chung network namespace với host, giúp giảm độ trễ mạng nhưng ít tính cô lập.
- **Overlay:** Kết nối các container giữa nhiều host khác nhau, thường dùng trong Docker Swarm hoặc hệ thống phân tán.

- **Macvlan:** Gán địa chỉ MAC thật cho container, giúp nó xuất hiện như một thiết bị vật lý trên mạng.
- **None:** Tắt toàn bộ chức năng mạng, container hoàn toàn biệt lập.

Ngoài ra, Docker cũng hỗ trợ tích hợp driver tùy chỉnh thông qua plugin system từ bên thứ ba. Các plugin như Calico hoặc Weave Net cung cấp các tính năng nâng cao như network policies, encryption và multi-cloud networking giúp mở rộng chức năng mạng của Docker.

Cách tạo và quản lý Docker Network hiệu quả. Docker cung cấp nhiều tùy chọn linh hoạt thông qua dòng lệnh (CLI), cho phép người dùng thiết lập cấu trúc mạng phù hợp với từng loại ứng dụng và mục tiêu triển khai.

Trong môi trường production, việc sử dụng user-defined network thay vì default bridge network được khuyến khích vì mang lại: Isolation tốt hơn giữa các container, tự động phân giải DNS và kết nối linh hoạt hơn.

Cách tạo docker network như sau:

```
$docker network create --driver bridge my_custom_network
```

Trong đó: `--driver` chỉ định loại docker network driver (ví dụ: bridge, host, overlay...); `my_custom_network` là tên mạng do người dùng đặt.

Ngoài ra, ta cũng có thể thêm các tùy chọn như subnet, gateway hoặc IP range để kiểm soát tốt hơn cấu hình mạng:

```
$docker network create --driver bridge --subnet 192.168.100.0/24 \
--gateway 192.168.100.1 --ip-range 192.168.100.128/25 \
--dns 8.8.8.8 --dns 8.8.4.4 --label environment=production \
--label project=myapp my_advanced_network
```

Một số lệnh thường dùng để quản lý network trong docker như:

<code>\$docker network ls</code>	<i># Liệt kê networks</i>
<code>\$docker network create mynet</code>	<i># Tạo network</i>
<code>\$docker network inspect bridge</code>	<i># Xem chi tiết</i>
<code>\$docker network connect mynet myapp</code>	<i># Kết nối container vào network</i>
<code>\$docker network rm mynet</code>	<i># Xóa network</i>

Ví dụ sau đây được sử dụng để kết nối các container qua network:

```
# Tạo network
$docker network create backend-net

# Chạy database
$docker run -d --name db --network backend-net -e POSTGRES_PASSWORD=123 postgres:16

# Chạy API kết nối với database
docker run -d --name api \
--network backend-net \
-p 3000:3000 \
-e DB_HOST=db \
myapi:1.0
```

Container api có thể truy cập database qua hostname db mà không cần biết IP.
Những khuyến nghị khi sử dụng volume và network trong docker:

- **Volume:**
 - Sử dụng Named Volume cho production thay vì Bind Mount.
 - Backup volume định kỳ (docker volume hoặc công cụ bên thứ ba).
 - Không mount thư mục nhạy cảm không cần thiết.
- **Network:**
 - Đặt từng nhóm service vào network riêng (principle of least privilege).
 - Tránh dùng network mặc định bridge cho production.
 - Kết hợp với firewall (Docker + UFW hoặc cloud security group).
 - Sử dụng Docker Secrets hoặc Swarm Secrets để truyền thông tin nhạy cảm.

Bên cạnh đó: Volume giúp tách biệt dữ liệu, dễ áp dụng encryption. Trong khi network isolation giúp giới hạn phạm vi tấn công (nếu một service bị hacker tấn công, khó lan rộng).

Volume và Network là hai thành phần then chốt giúp Docker chuyển từ “chạy thử nghiệm” sang “triển khai production”. Thành thạo hai khái niệm này sẽ giúp chúng ta dễ dàng làm việc với Docker Compose và sau này là Kubernetes.

2.2.4 Xây dựng và đóng gói ứng dụng web qua docker

Một trong những ứng dụng thực tế và phổ biến nhất của Docker là đóng gói và triển khai các ứng dụng Web. Docker giúp chúng ta đóng gói toàn bộ ứng dụng (code, web server, runtime, thư viện, và database) thành một hoặc nhiều container, đảm bảo tính nhất quán giữa các môi trường Development – Testing – Production.

Lợi ích nổi bật khi Dockerize ứng dụng Web:

- Không còn lỗi “chạy tốt trên máy tôi”.
- Triển khai nhanh chóng và lặp lại được.
- Dễ scale và quản lý microservices.
- Tăng cường bảo mật qua isolation và user non-root.
- Dễ tích hợp CI/CD.

Các bước chung để đóng gói ứng dụng Web:

- **Bước 1:** Chúng ta cần lựa chọn Base Image phù hợp, một số image phổ biến và phù hợp với các ngôn ngữ lập trình web thường dùng như:
 - PHP → `php:8.3-apache` hoặc `php:8.3-fpm` + `nginx`
 - Node.js → `node:20-alpine`
 - Python → `python:3.12-slim`
 - Java → `openjdk:21-slim`
 - Tùy phiên bản cần sử dụng mà ta có thể tùy biến.
- **Bước 2:** Cài đặt dependencies (`RUN apt-get / apk add / composer install...`)
- **Bước 3:** Copy source code vào thư mục web root (`/var/www/html` cho Apache).
- **Bước 4:** Cấu hình môi trường (ENV, config files).
- **Bước 5:** Expose port và định nghĩa lệnh chạy (CMD / ENTRYPOINT).
- **Bước 6:** Sử dụng Multi-stage build (nếu cần tối ưu).

- **Bước 7:** Nâng cao hơn, chúng ta có thể kết hợp Docker Compose để quản lý multi-container (Web + Database + Cache...).

Kiến trúc phổ biến:

- Monolith: 1 container (PHP + Apache)
- Multi-container: Web + DB + Redis (khuyến nghị dùng trên production)

Trong ví dụ sau đây, chúng ta sẽ thực hiện đóng gói ứng dụng Web PHP đơn giản với docker. Giả sử ta đã xây dựng một ứng dụng PHP đơn giản có chức năng:

- Hiện thị danh sách sinh viên từ database MySQL.
- Thêm sinh viên mới (form cơ bản).

Cấu trúc thư mục dự án như sau:

```
php-web-app/  
├─ Dockerfile  
├─ src/  
│   ├── index.php  
│   ├── add.php  
│   └─ config.php  
├─ data/  
│   └─ init.sql  
└─ .dockerignore
```

Hình 2.9: Cấu trúc thư mục của dự án web sử dụng PHP

Phần mã nguồn của ứng dụng sẽ không đề cập đến ở đây vì ngoài phạm vi nội dung môn học, chúng ta sẽ tham khảo nội dung mã nguồn của dockerfile dùng để tạo container cho việc đóng gói ứng dụng:

```
# Dockerfile  
FROM php:8.3-apache  
  
# Cài đặt các extension PHP cần thiết  
RUN docker-php-ext-install pdo pdo_mysql mysqli  
  
# Copy source code vào web root của Apache  
COPY src/ /var/www/html/  
  
# Phân quyền cho thư mục web  
RUN chown -R www-data:www-data /var/www/html \  
&& chmod -R 755 /var/www/html  
  
# Bật mod rewrite (nếu sau này dùng .htaccess)  
RUN a2enmod rewrite
```

Chương 2. Triển khai ứng dụng qua Docker

```
# Expose port
EXPOSE 80

# Chạy Apache ở foreground
CMD ["apache2-foreground"]
```

Triển khai bằng lệnh Docker:

Bước 1: Tạo Docker Network

```
$docker network create php-network
```

Bước 2: Chạy MySQL Container

```
$docker run -d \
  --name mysql-db \
  --network php-network \
  -e MYSQL_ROOT_PASSWORD=Password123 \
  -e MYSQL_DATABASE=university \
  -v mysql_data:/var/lib/mysql \
  -v $(pwd)/data/init.sql:/docker-entrypoint-initdb.d/init.sql \
  mysql:8.0
```

Bước 3: Thực hiện build PHP Image. Thực thi lệnh:

```
$docker build -t php-web-app:1.0 .
```

Lưu ý: dấu “.” trong câu lệnh để docker tìm và build Image từ file trong thư mục hiện hành.

Bước 4: Chạy PHP Web Container. Thực hiện lệnh:

```
$docker run -d \
  --name php-web \
  --network php-network \
  -p 8080:80 \
  php-web-app:1.0
```

Bước 5: Kiểm tra và Debug. Thực hiện lệnh:

```
# Xem danh sách container đang chạy
$docker ps
# Xem log của web
$docker logs php-web
# Xem log của database
$docker logs mysql-db
# Vào bên trong container PHP để test
$docker exec -it php-web bash
# Kiểm tra kết nối mạng
$docker exec -it php-web ping mysql-db
```

Truy cập ứng dụng: Mở trình duyệt và truy cập <http://localhost:8080>.

Bài thực hành

Bài 1

Hãy thực hiện các lệnh để quản lý container, image, volume và network trong Docker.

Bài 2

Hãy sửa file dockerfile trong ứng dụng cuối bài để sử dụng phiên bản mới nhất của PHP, chuyển sang cổng 8889 cho ứng dụng web và thay đổi tag cho container. Sau khi hoàn tất, hãy build lại Image và khởi tạo lại container, kiểm tra log và trạng thái của các container khi hoạt động.